Laboratorio: arquitectura de estado reactivo (patrón Store)

App `tech-ticket` · Ionic 9 + Angular 22 standalone · Signals + HttpClient
Jorge González Gómez · 2º DAM · Desarrollo de Interfaces (0488) · Universidad Nebrija · versión 2, 26-10-2026
Código: github.com/JorgeGonzalezDI/Desarrollo-Interfaces-DAM-2/tree/main/05-Lab-Reactividad-TechTicket

Práctica completa. Un **Store** de estado (servicio con signals) guarda los tickets, si está cargando y el error; el componente solo es la Vista. Filtro Todos/Abiertos/Cerrados en memoria, sin volver a pedir los datos. Código comentado en GitHub.

App general de repaso. Para entender bien los conceptos de las prácticas 04–08 sin rehacerlas una a una, estoy haciendo una sola app de gestión con roles que junta lo importante de cada una (Supabase, guards, formularios reactivos y estado con signals). Está en el mismo repositorio: rama `app-gestion-roles`, carpeta `09-App-Gestion-Roles`.

1. Estructura del proyecto

Proyecto Angular standalone (sin NgModules) con Ionic, con el servicio Store (`core/services`) y el componente del panel (`features/ticket-dashboard`):

```
tech-ticket/
|-- capacitor.config.ts        (appId: io.ionic.techticket, webDir: www)
`-- src/app/
    |-- app.config.ts                                    (provideHttpClient, provideIonicAngular)
    |-- core/
    |   |-- models/ticket.model.ts                        interfaz Ticket (id, title, completed, userId)
    |   `-- services/ticket.service.ts                     <- Store de estado (signals)
    `-- features/ticket-dashboard/ticket-dashboard.component.ts   <- Vista (OnPush)
```

2. Arquitectura: patrón Store (MVVM) + SOLID

`TicketService` es la fuente única de la verdad: guarda en un signal privado `state` los datos, si está cargando y el error. El componente solo es la Vista y lee esas señales, así la lógica HTTP queda separada de la interfaz.

- **SRP:** el servicio solo gestiona el estado de los tickets; el componente solo pinta y recoge eventos (filtros, reintentar).
- **OCP:** el estado es `private` y se expone en solo lectura con `computed()` (`tickets`, `isLoading`, `errorMessage`).
- **DIP:** dependencias con `inject()` (`HttpClient`), sin `new`, lo que facilita los tests con mocks.
- **Vista:** `OnPush`, signal local `filtroActual` y `computed ticketsFiltrados`, que se recalcula si cambian los tickets o el filtro; `@if/@else if/@for track` para carga, error y lista.

3. El Store: código clave

```
// ticket.service.ts
private state = signal<TicketState>({ tickets: [], loading: true, error: null });

readonly tickets       = computed(() => this.state().tickets);       // solo lectura
readonly isLoading     = computed(() => this.state().loading);
readonly errorMessage  = computed(() => this.state().error);

fetchTickets(): void {
  this.state.update((s) => ({ ...s, loading: true, error: null }));
  this.http.get<Ticket[]>(this.apiUrl).subscribe({
    next:  (tickets) => this.state.set({ tickets, loading: false, error: null }),
    error: () => this.state.set({ tickets: [], loading: false, error: 'No se han podido cargar los
tickets.' }),
  });
}
```

El filtro se calcula en el componente, sin tocar el Store ni volver a llamar a la API:

```
// ticket-dashboard.component.ts
protected readonly ticketsFiltrados = computed(() => {
  const tickets = this.ticketService.tickets();
  switch (this.filtroActual()) {
    case 'abiertos': return tickets.filter((t) => !t.completed);
    case 'cerrados': return tickets.filter((t) => t.completed);
    default:         return tickets;
  }
});
```

4. Funcionamiento

Todos: los 15 tickets de la API (jsonplaceholder.typicode.com/todos) con badge Pendiente / Resuelto. **Abiertos:** `filtroActual.set('abiertos')` → solo `completed = false`. **Cerrados:** solo `completed = true`. Filtrado en memoria, sin volver a llamar a la API.
Carga: mientras `isLoading()` es `true` se ve el spinner «Sincronizando incidencias...». **Error de red:** el `subscribe` entra en `error`, se muestra el aviso y «Reintentar» vuelve a llamar a `fetchTickets()`.

5. Empaquetado en Android con Capacitor

`ionic build --prod` genera el bundle en `www/`. `npx cap add android` crea la carpeta `android/` y `npx cap sync android` copia el build a `android/app/src/main/assets/public` (hay que repetirlo tras cada build).
Pendiente: abrirlo en Android Studio con `npx cap open android` y probarlo en el emulador, o con `npx cap run android` en un móvil por USB.

6. Ejecutar

En `05-Lab-Reactividad-TechTicket/tech-ticket`: `npm install` y `ionic serve`.